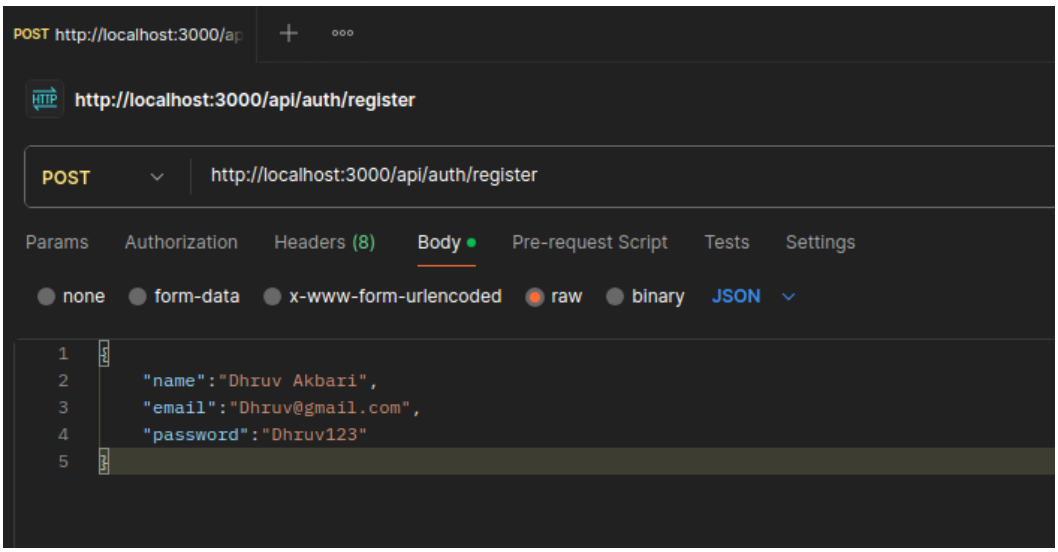
Node.Js Practical Assignment - 2

Name : Akbari Dhruv
Roll no : 01
Div : ICT-3

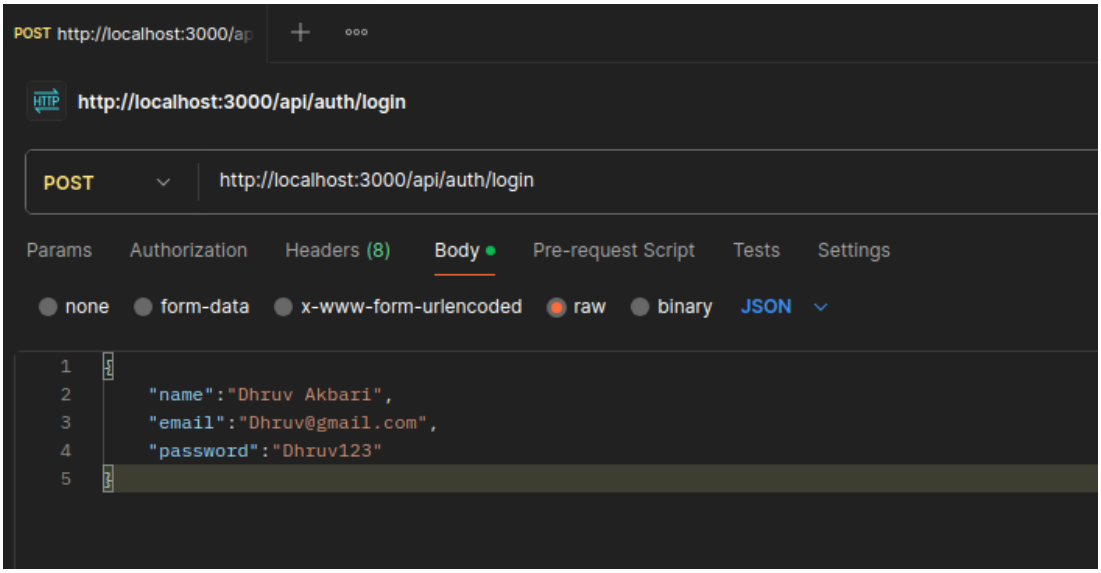
1. **Blog Management REST API with Authentication (Node.js, Express, MongoDB, EJS) Scenario: A startup wants to build the backend and a basic server- rendered interface for a personal blogging platform where registered users can sign up, log in, and manage their own blog posts. Only the author of a post (or an admin) should be able to edit or delete it. You have been asked to build this system. Build an application using Node.js, Express.js, MongoDB, Mongoose, and EJS (for rendering views) that implements the following:**

1.User Authentication

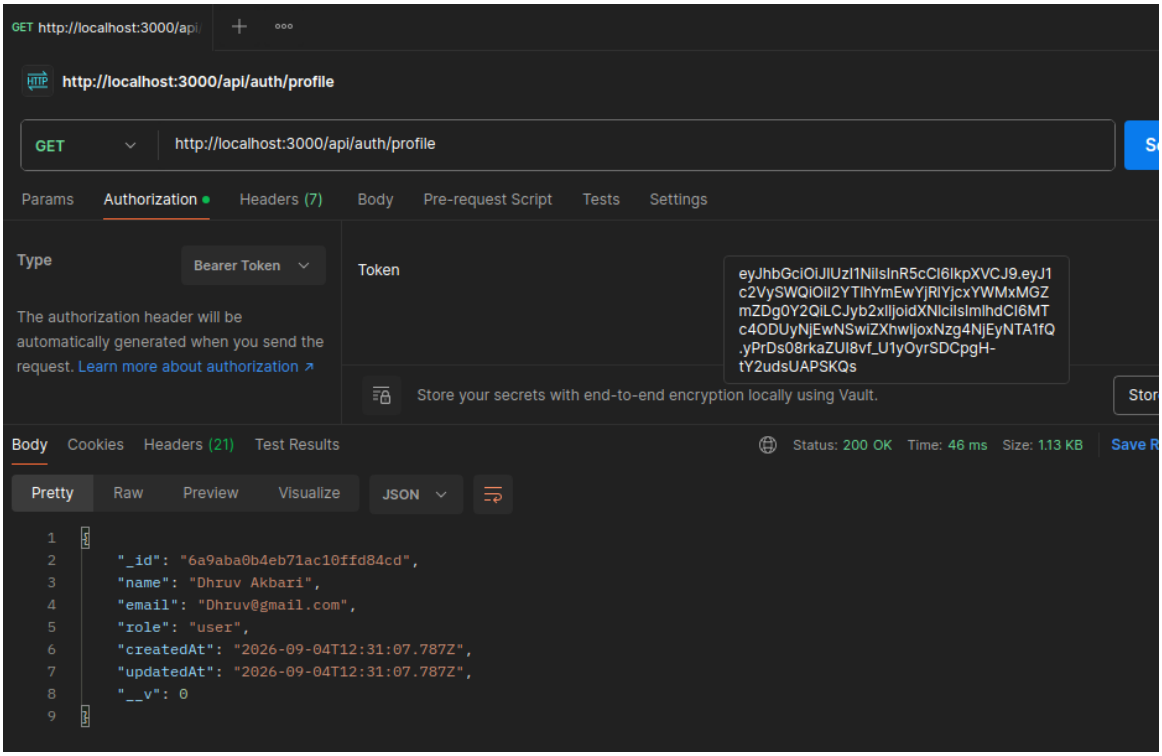
• POST /api/auth/register – Register a new user (name, email, password) with the password hashed using bcrypt.



• POST /api/auth/login – Authenticate a user and return a JWT token.

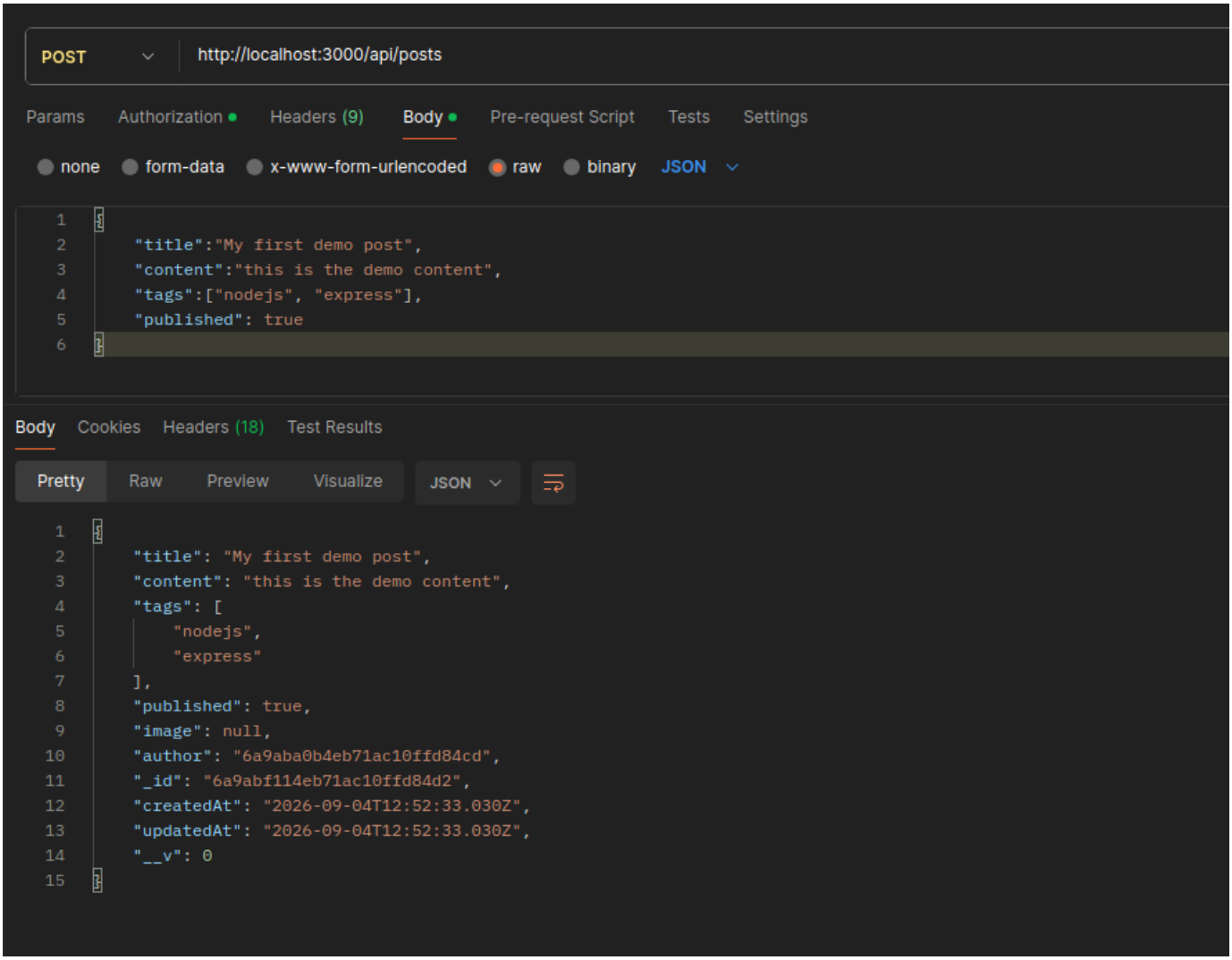


• GET /api/auth/profile – A protected route that returns the logged-in user's Profile.

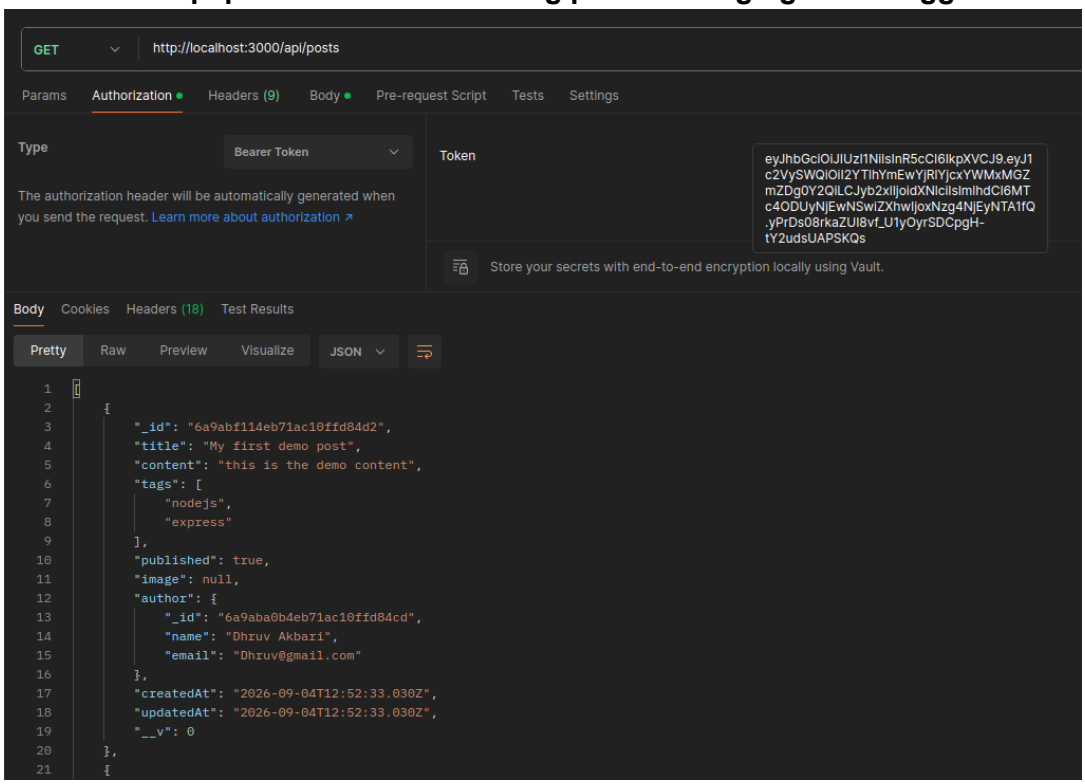


2. Blog Post CRUD API (protected routes, linked to the logged-in author)

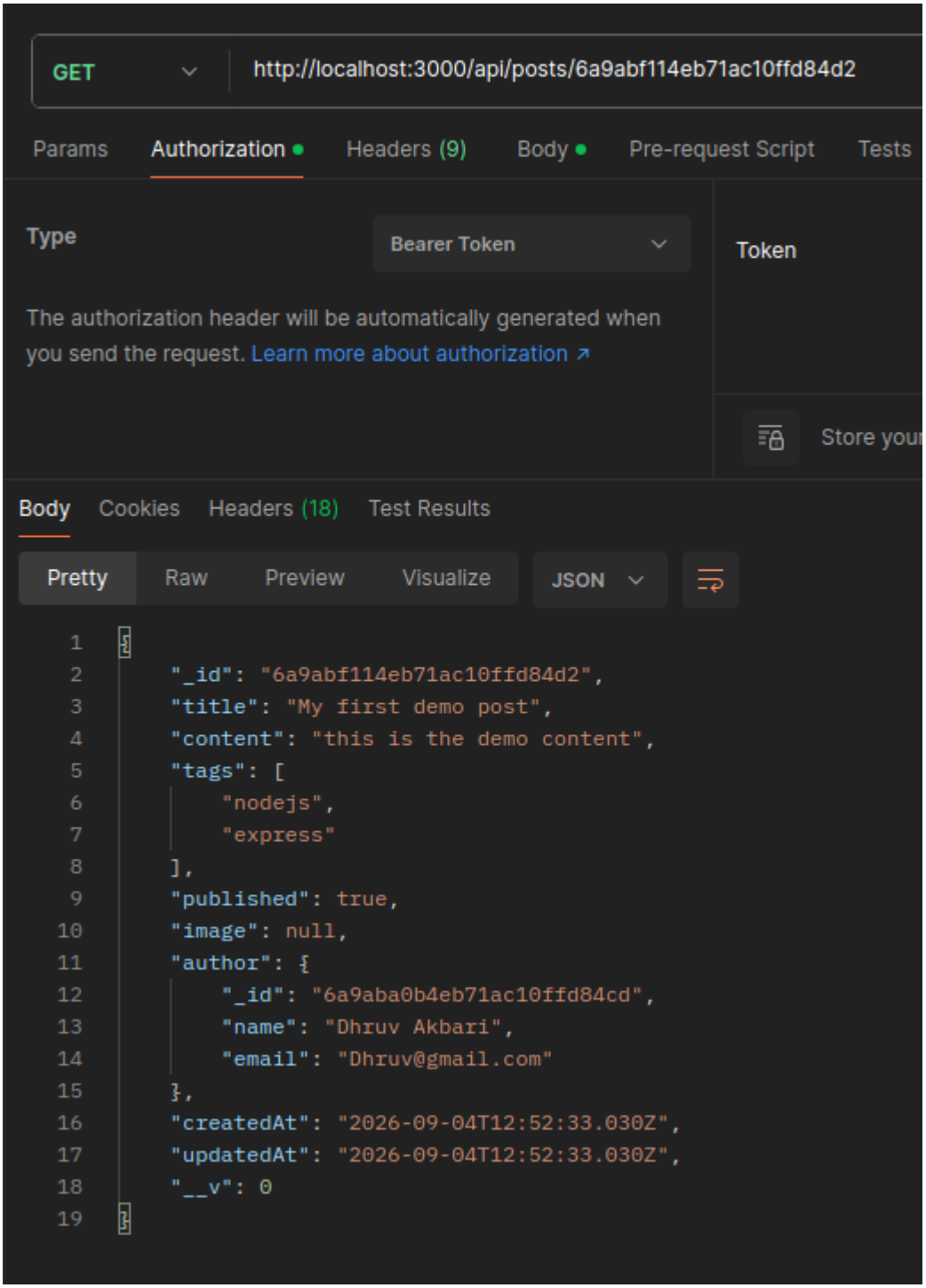
- **POST /api/posts – Create a new blog post (title, content, tags, published status).**


POST
http://localhost:3000/api/posts
Params
Authorization
Headers (9)
Body
Pre-request Script
Tests
Settings
none
form-data
x-www-form-urlencoded
raw
binary
JSON
{
"title":"My first demo post",
"content":"this is the demo content",
"tags":["nodejs", "express"],
"published": true
}
Body
Cookies
Headers (18)
Test Results
Pretty
Raw
Preview
Visualize
JSON
{
"title": "My first demo post",
"content": "this is the demo content",
"tags": [
"nodejs",
"express"
],
"published": true,
"image": null,
"author": "6a9aba0b4eb71ac10ffd84cd",
"_id": "6a9abf114eb71ac10ffd84d2",
"createdAt": "2026-09-04T12:52:33.030Z",
"updatedAt": "2026-09-04T12:52:33.030Z",
"__v": 0
}

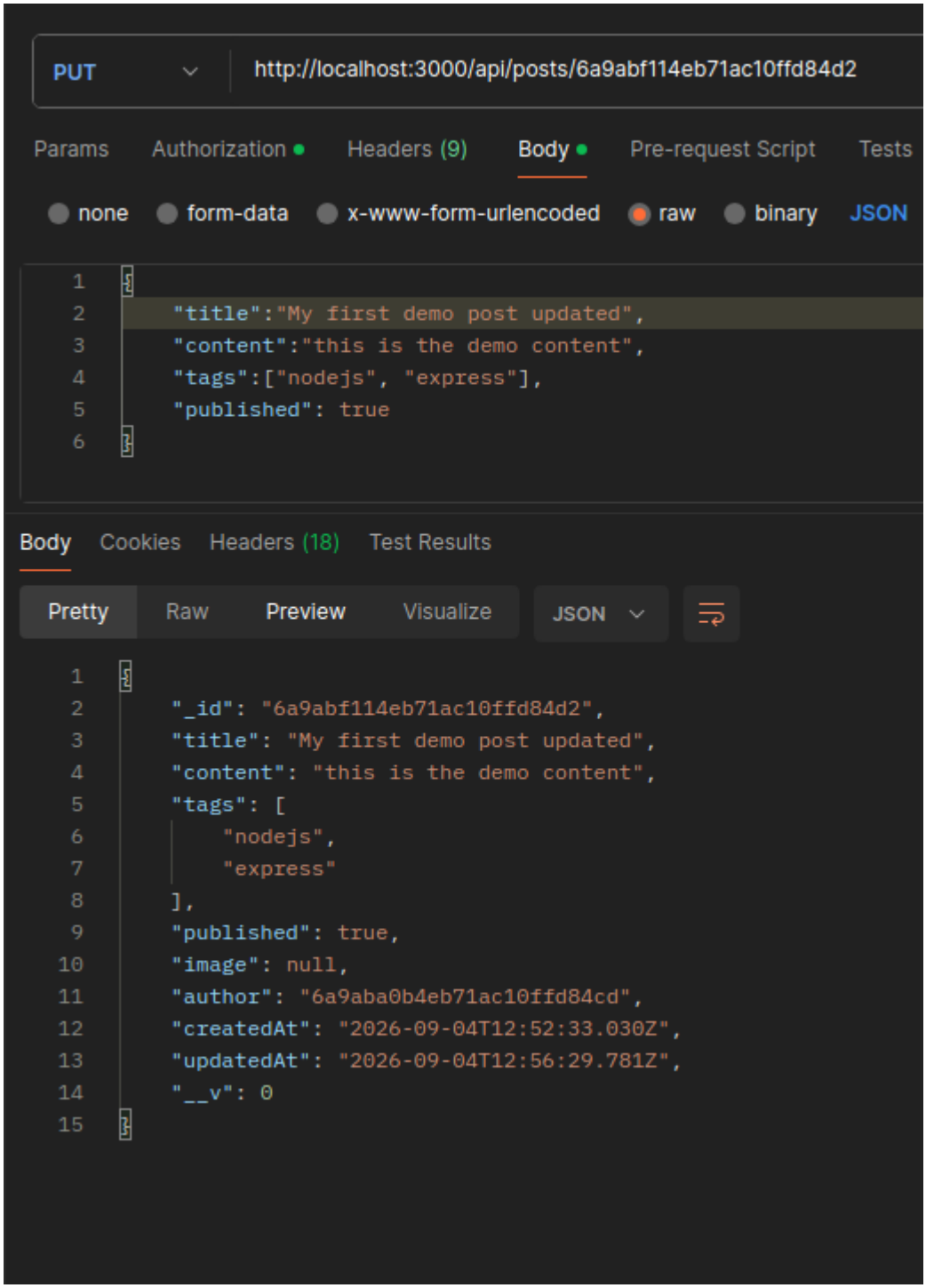
• GET /api/posts – Retrieve all blog posts belonging to the logged-in user.



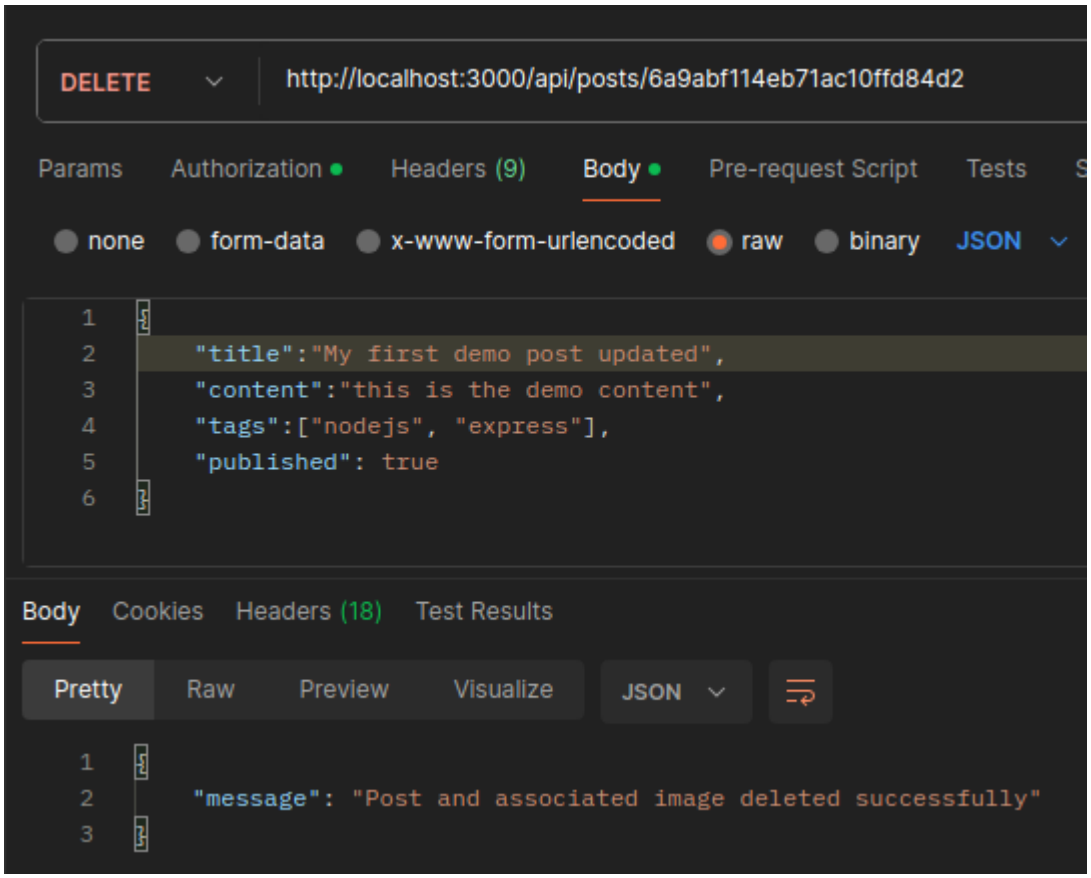
- **GET /api/posts/:id – Retrieve a single blog post by ID.**



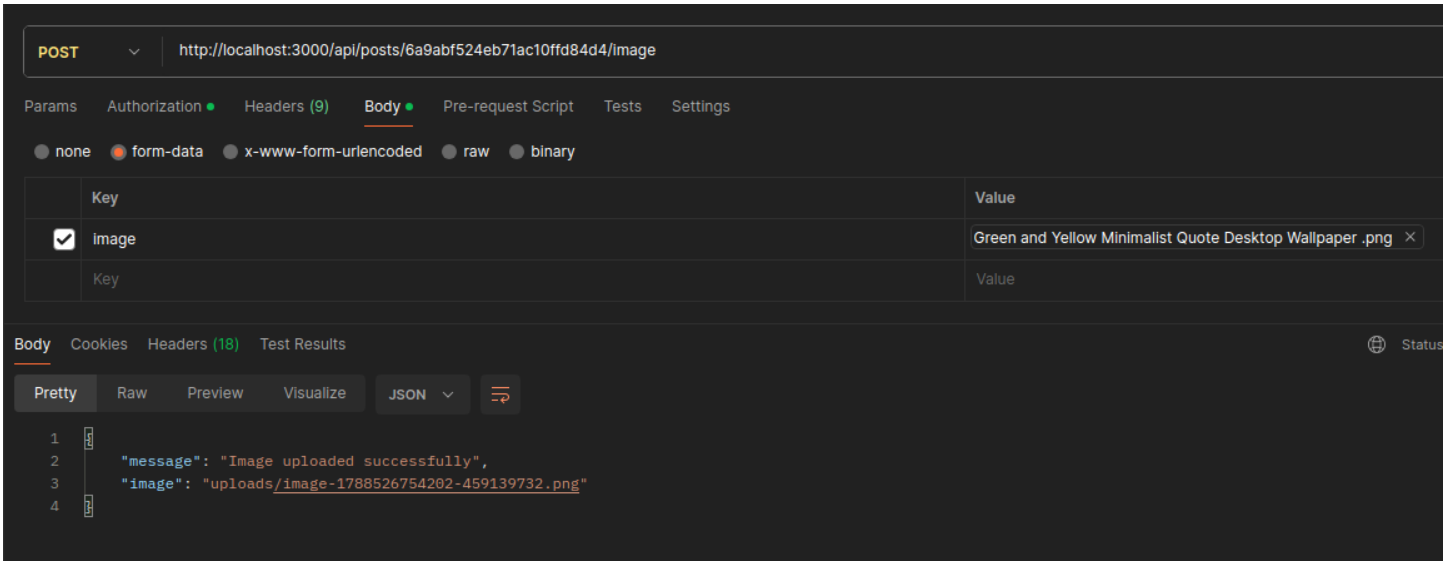
• PUT /api/posts/:id – Update a blog post by ID (only the author or an admin can update).



• DELETE /api/posts/:id – Delete a blog post by ID (only the author or an admin can delete).

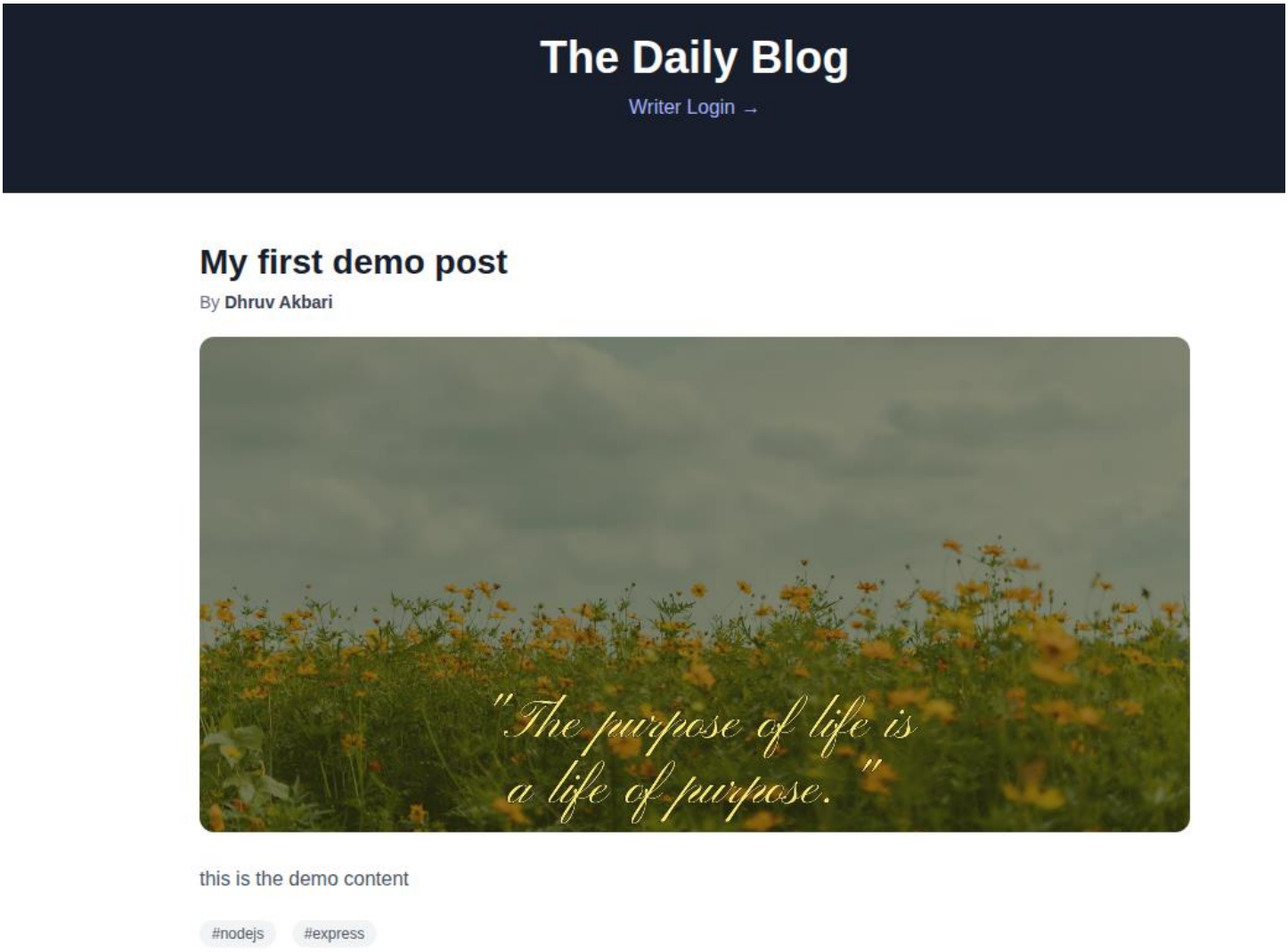


• POST /api/posts/:id/image – Upload or replace a featured image for a blog post (multipart/form-data) using Multer.

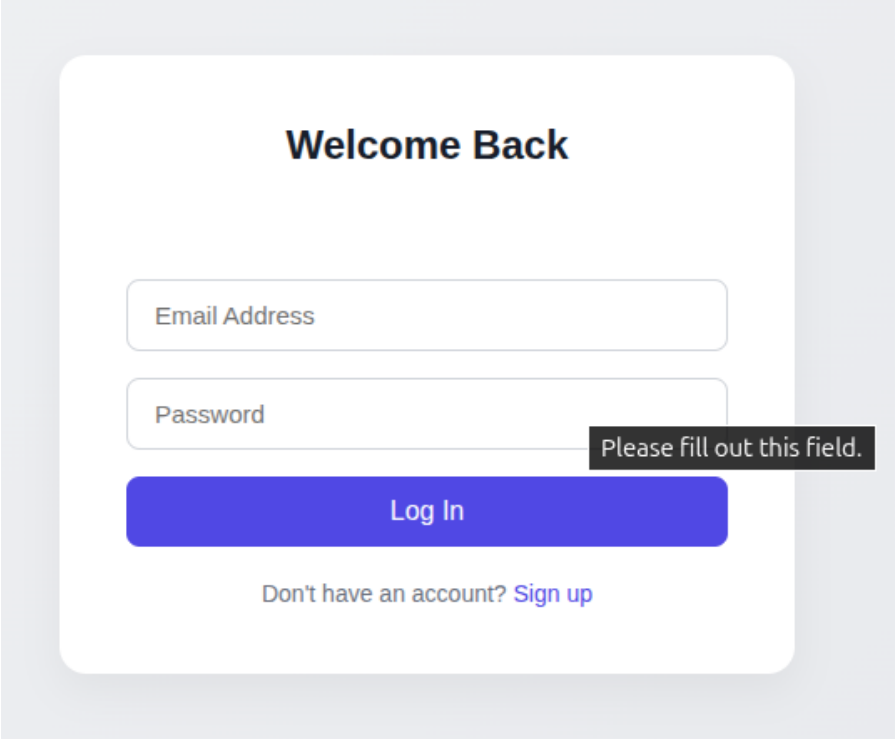


3. EJS Views

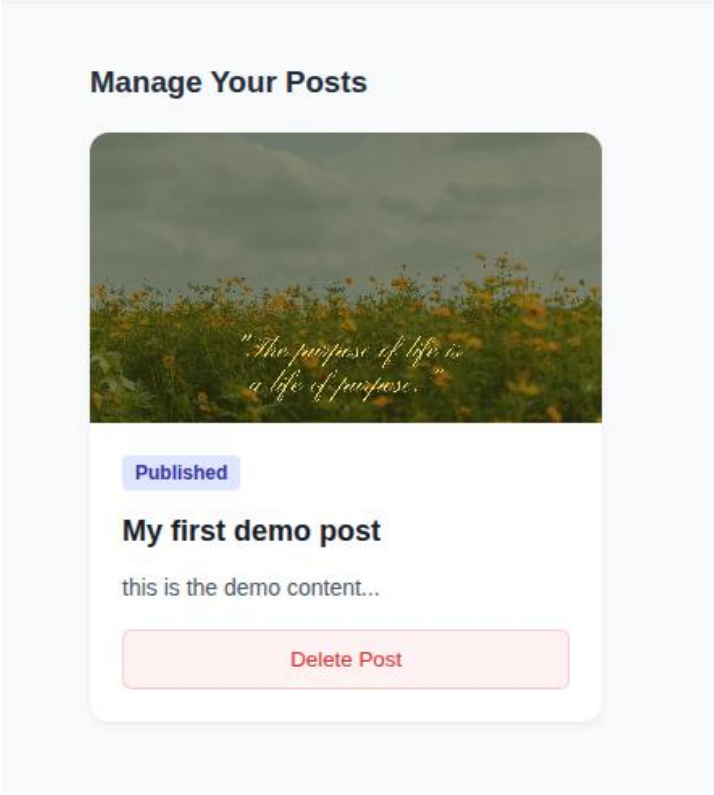
• Render an EJS page (views/posts.ejs) that displays all published blog posts in a styled list.



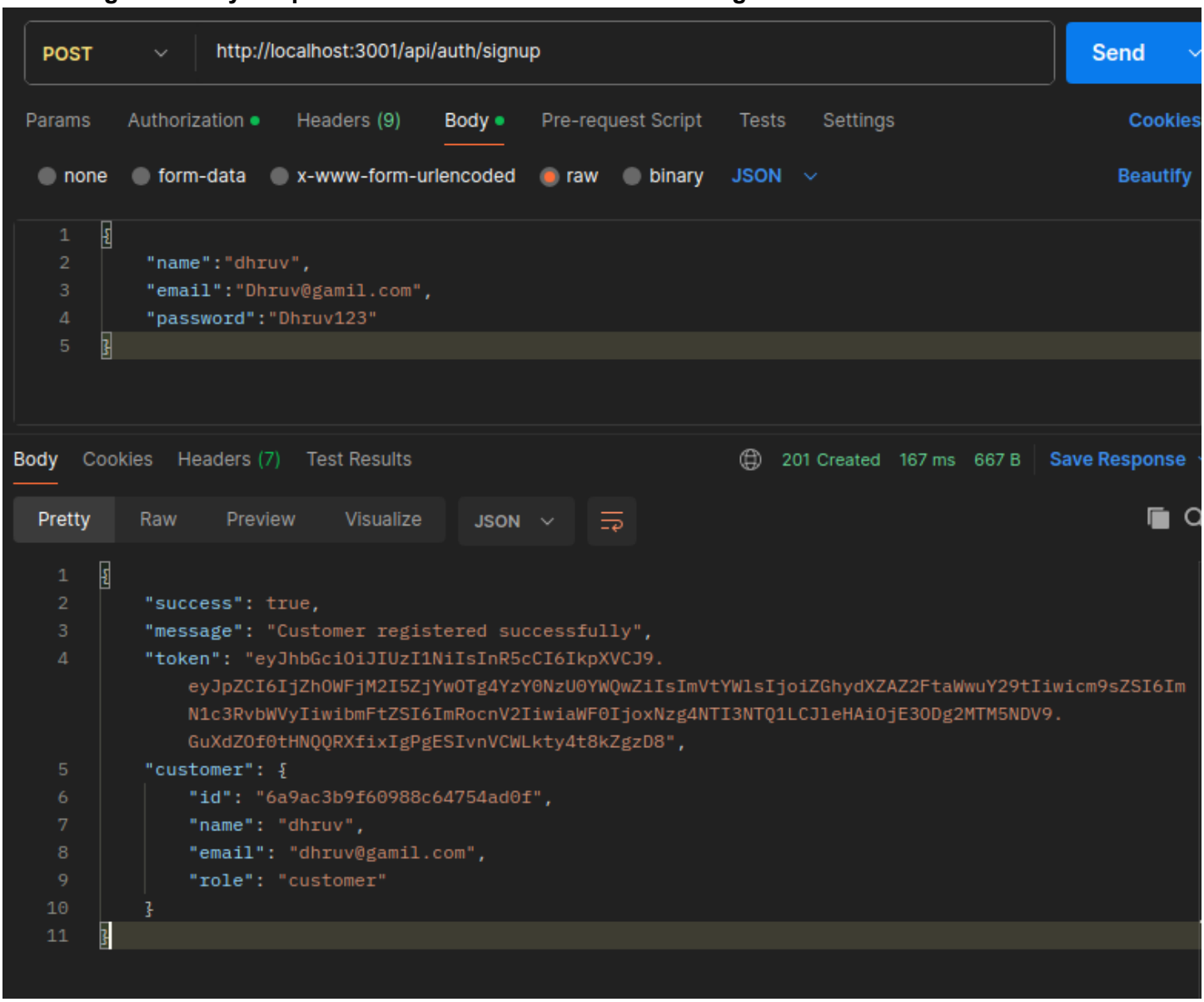
• Render an EJS page (views/login.ejs) with a login form that submits to your login route.

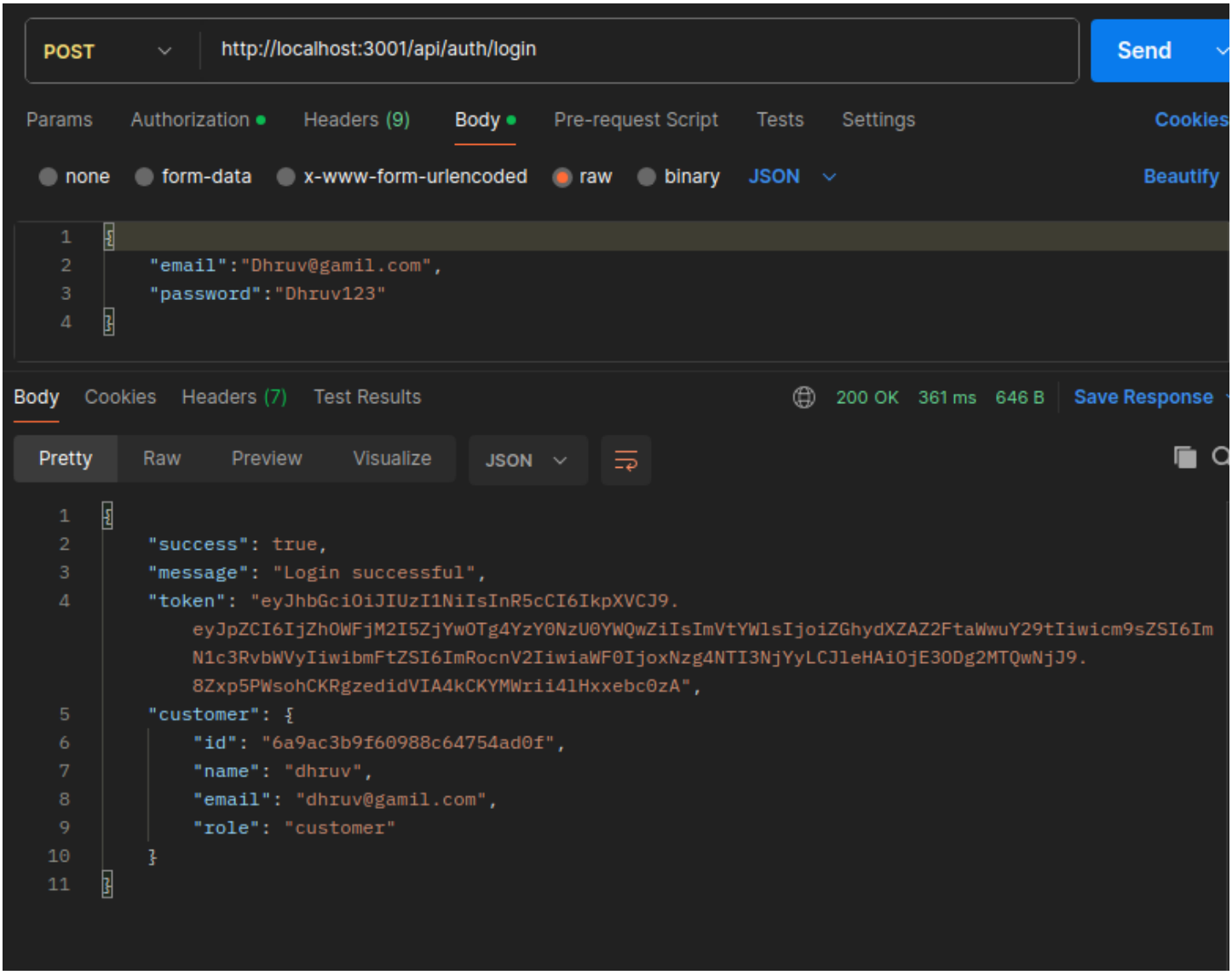


• Render an EJS page (views/dashboard.ejs, protected) showing the logged-in user's own posts with edit/delete links, visible only after login.

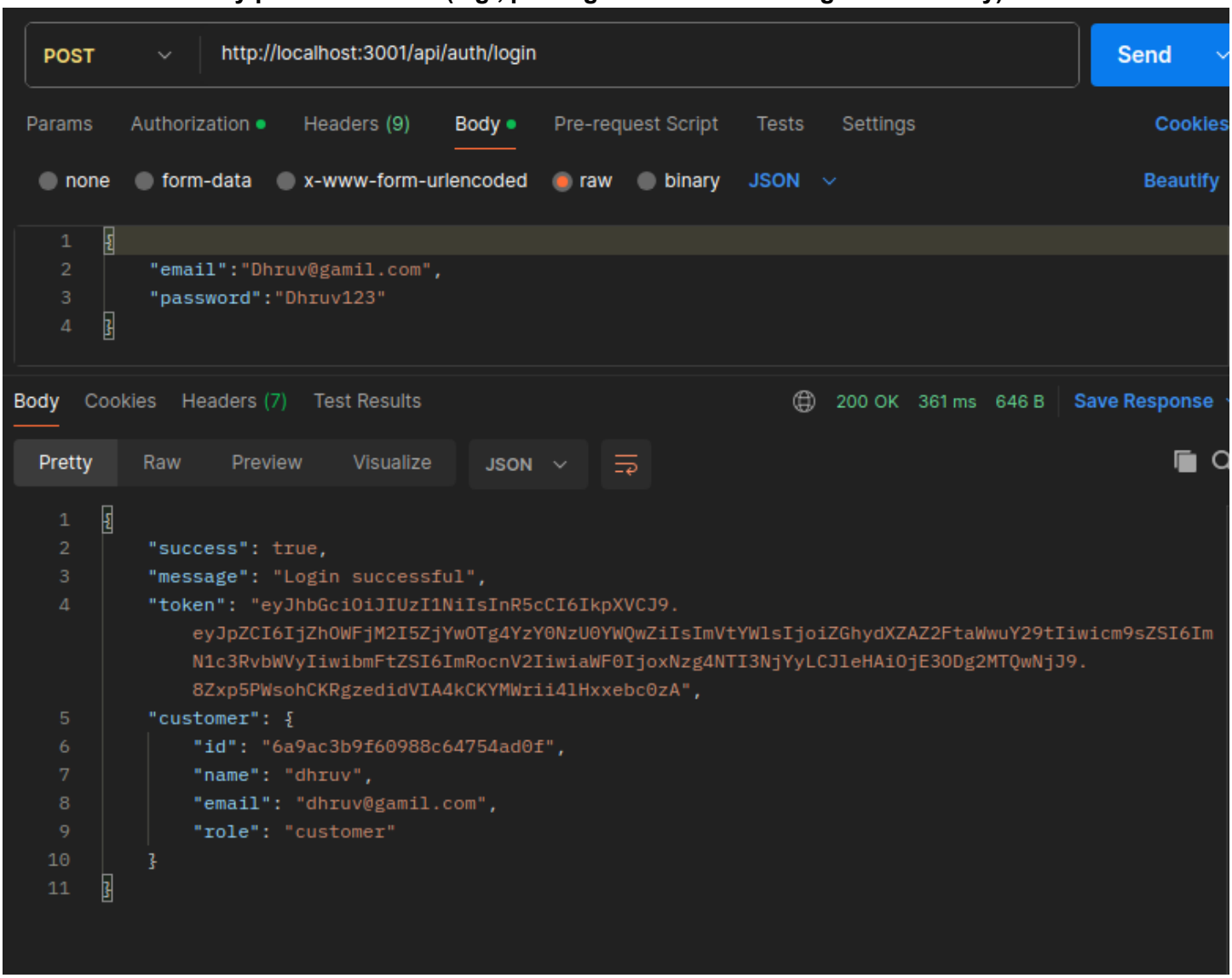


2. **Secure Signup for Bookstore Customers Scenario: A new customer is creating an account on the online bookstore to start placing orders for books. Task: Write a program that hashes the customer's password using bcrypt before storing it in MongoDB, and write the corresponding logic to verify the password when the same customer logs in later.**

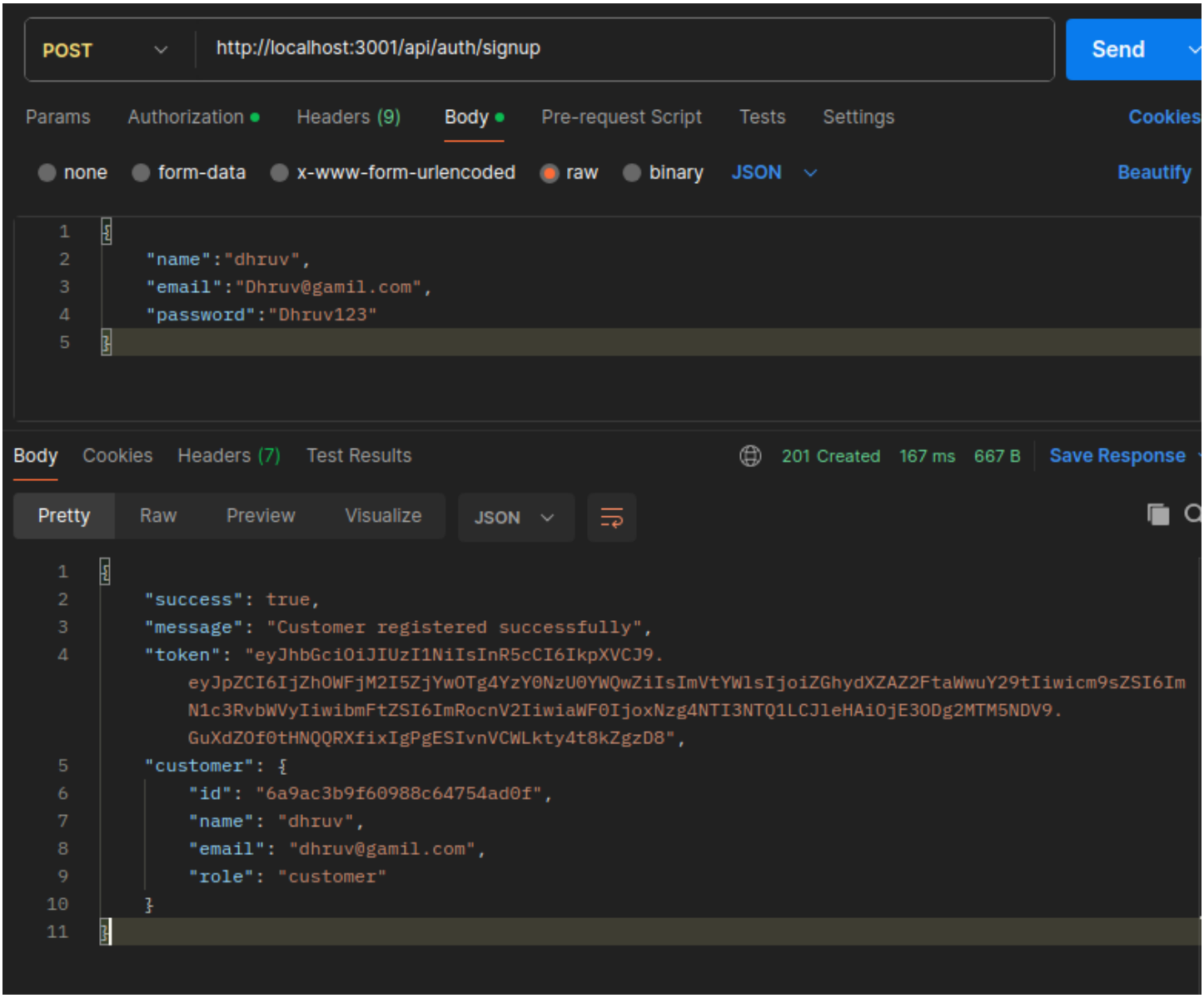



POST
http://localhost:3001/api/auth/login
Send
Params
Authorization
Headers (9)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
JSON
Beautify
{
"email":"Dhruv@gamil.com",
"password":"Dhruv123"
}
Body
Cookies
Headers (7)
Test Results
200 OK
361 ms
646 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"success": true,
"message": "Login successful",
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJpZCI6IjZhOWFjM2I5ZjYwOTg4YzY0NzU0YWQwZiIsImVtYWlsIjoiZGhydXZAZ2FtaWwuY29tIiwicm9sZSI6Im
N1c3RvbWVyIiwibmFtZSI6ImRocnV2IiwiaWF0IjoxNzg4NTI3NjYyLCJleHAiOjE3ODg2MTQwNjJ9.
8Zxp5PWsohCKRgzedidVIA4kCKYMWrii4lHxxebc0zA",
"customer": {
"id": "6a9ac3b9f60988c64754ad0f",
"name": "dhruv",
"email": "dhruv@gamil.com",
"role": "customer"
}
}

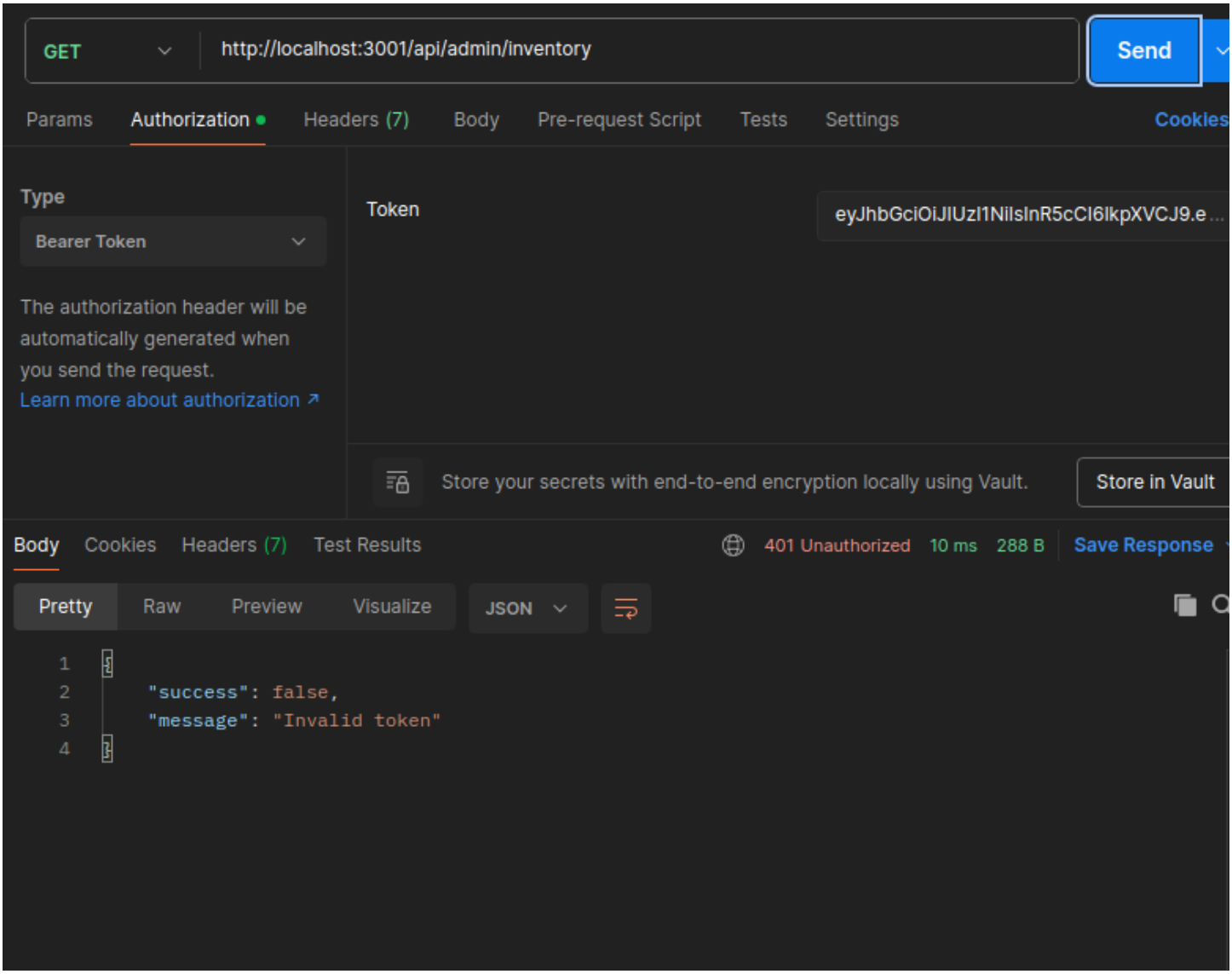
3. **Issuing a Digital Membership Token on Login Scenario: Once a customer logs in successfully, the store must issue them a token (their digital membership pass) so they don't need to log in again for every request, such as viewing their orders or checking out. Task: Write a program to generate a JWT token on successful login, and a middleware to verify this token before allowing access to any protected route (e.g., placing an order or viewing order history).**



4. **Defining Customer and Admin Roles Scenario: The bookstore platform has two kinds of accounts regular Customers who browse and buy books, and Admins who manage inventory, pricing, and all customer orders. Task: Write a Mongoose User schema with a role field ('customer' / 'admin') that defaults to 'customer', so every new signup is treated as a regular customer unless explicitly promoted to admin.**



5. **Admin-Only Access to the Inventory Dashboard Scenario: The bookstore has an inventory management route (GET /api/admin/inventory) that lists stock levels and sales data for every book, meant only for store admins. Task: Write an authorization middleware that checks the logged- in user's role and returns 403 Forbidden if the user is not an 'admin', preventing regular customers from accessing this route.**



6. **Cancelling an Order — Customer or Admin Only Scenario: A customer should be able to cancel their own order before it ships. Separately, an admin should be able to cancel any customer's order for example, when a book goes out of stock. Task: Write a DELETE /api/orders/:id handler that allows an order to be cancelled only if the logged-in user is the customer who placed the order, or holds the admin role, and returns 403 Forbidden otherwise.**

